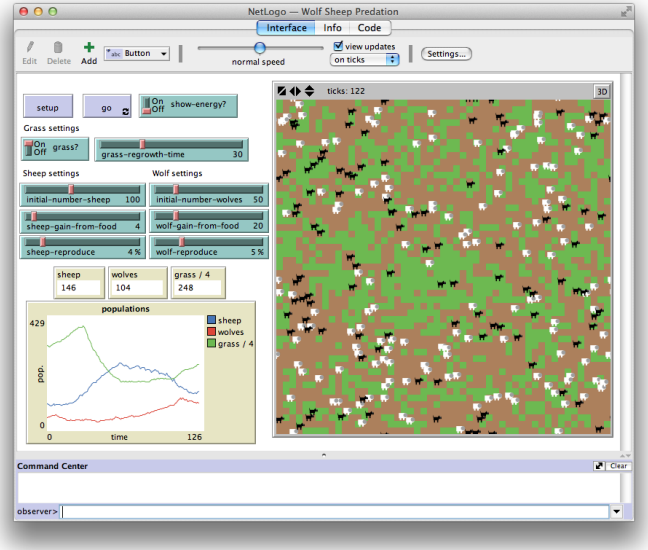
Foundations of Agent-based systems
Session 4 – Simulation

Nicolas Sabouret



Université Paris-Saclay

September 2026

Agenda

1. 2026/09/07 – Introduction and basics
   - 1:30 lecture: basics of MAS
   - 0:30 tutorial: MAS modeling
   - 1:00 practical session: MAS in Java
2. 2026/09/14 – Communication and platforms
   - 1:00 lecture: communication, platforms
   - 1:00 practical session: MAS in Java (cont.)
   - 1:00 tutorial : JADE
3. 2026/09/28 – Practical session : JADE
4. **2026/10/05 – MultiAgent Based Simulation**
   - 1:00 lecture: theory of MABS
   - 2:00 practical session: MESA
5. **2026/10/12 – Practical session : MABS project**
6. **2026/10/19 – Agentic AI (practical session)**

Session 4
MultiAgent Based Simulation

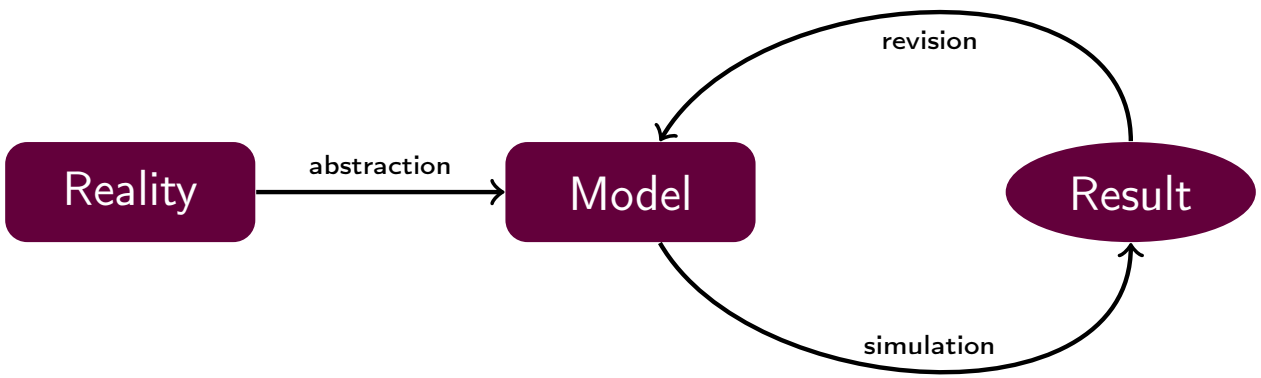
Session 4: agenda

- Simulation
- MultiAgent Based Simulation
- The Mesa platform
- PW : Mesa basics : the money model
- PW : Mesa visualization : solara or pygame interface

What is a simulation ?

Shannon, 1977

[...] designing a model of a real system
and conducting experiments with this model
for [...] understanding the behaviour of the system
or [...] evaluating various strategies [...] for the operation of the system



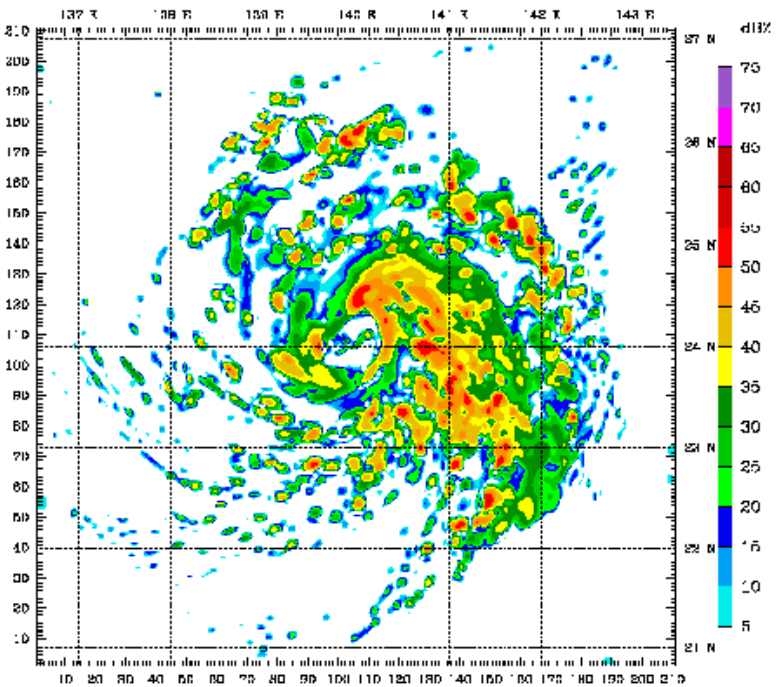
Goals

Test an hypothesis, study stable situations or resilience capabilities, *etc.*

Computer simulation

Iterative process (Fishwick, 1997)

- Design a model of a physical system
- Executing the model on a digital computer
- Analyzing the execution output

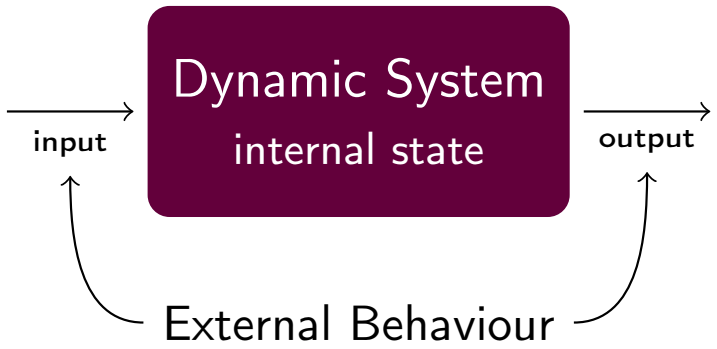


Dynamic systems

The majority of what we want to simulate are Dynamic Systems.

The black box problem

- ✓ Observable behaviour (reactions to inputs)
- ✗ Internal structure & dynamics



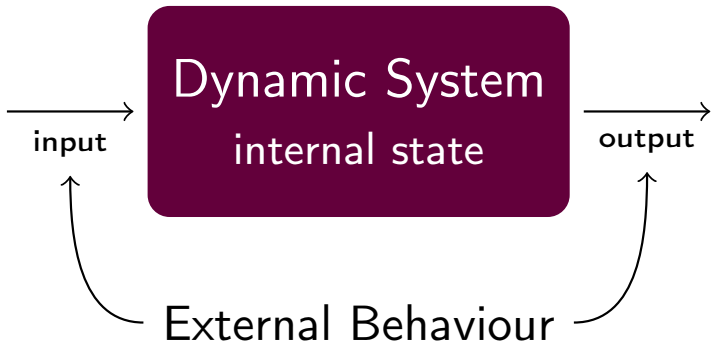
Note that an agent is a dynamic system too...

Dynamic systems

The majority of what we want to simulate are Dynamic Systems.

The black box problem

- ✓ Observable behaviour (reactions to inputs)
- ✗ Internal structure & dynamics



Note that an agent is a dynamic system too...

Problem

- ✓ Computer Simulation → Formal Specifications
- ✗ Lack of knowledge about the system's internal
- ✗ Choice of formalism constraints the observations

Methodology

Modeling & Simulation (M&S) theory

Methodology

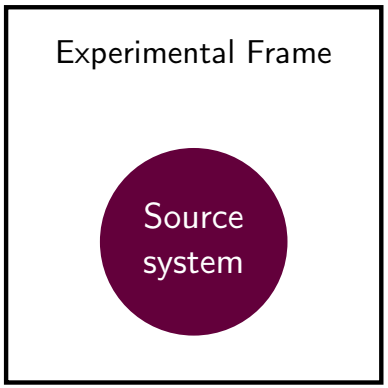
Modeling & Simulation (M&S) theory



1. What we want to model (source of observable data)

Methodology

Modeling & Simulation (M&S) theory



1. What we want to model (source of observable data)
2. What we want to study (goal of the simulation + conditions)

Methodology

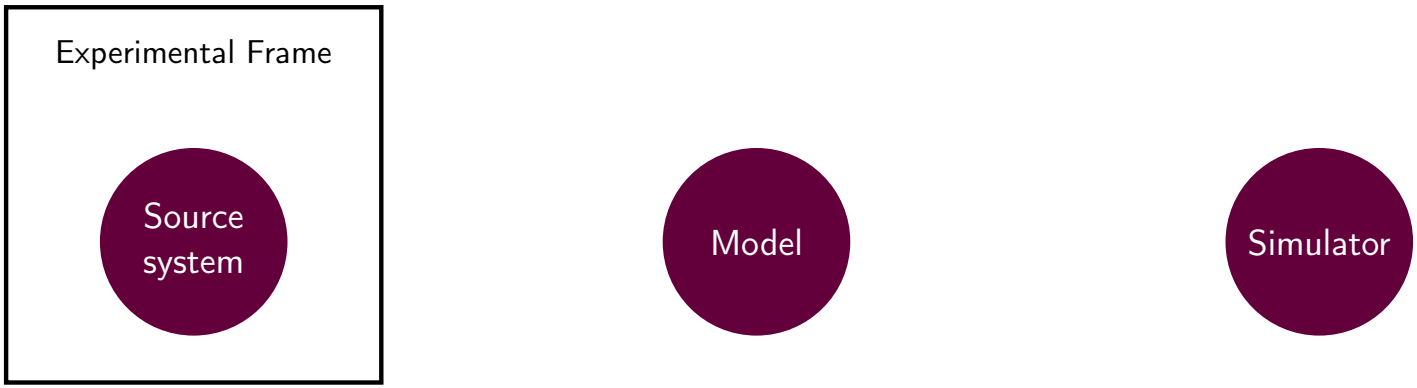
Modeling & Simulation (M&S) theory



1. What we want to model (source of observable data)
2. What we want to study (goal of the simulation + conditions)
3. Specifications to simulate the behaviour of the system

Methodology

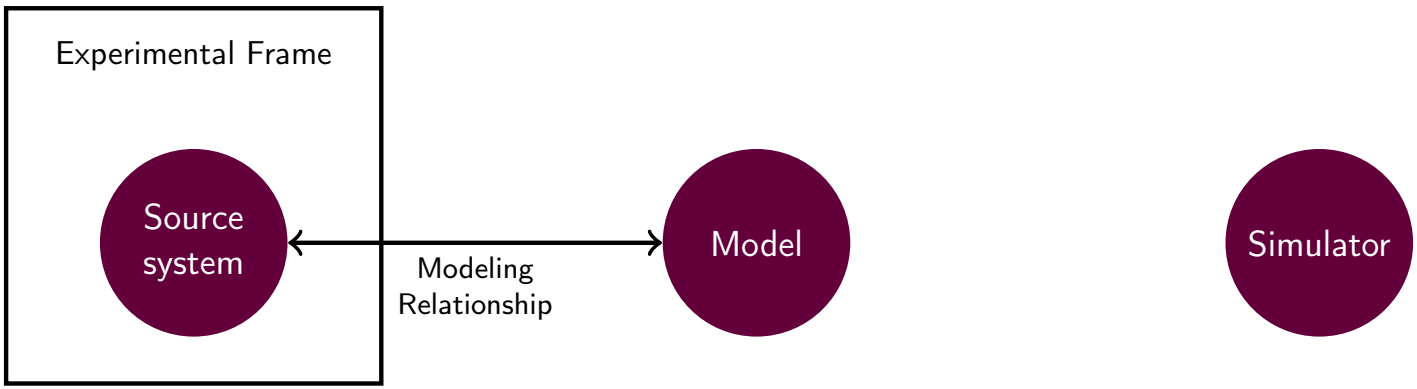
Modeling & Simulation (M&S) theory



1. What we want to model (source of observable data)
2. What we want to study (goal of the simulation + conditions)
3. Specifications to simulate the behaviour of the system
4. Computing system that can run the model

Methodology

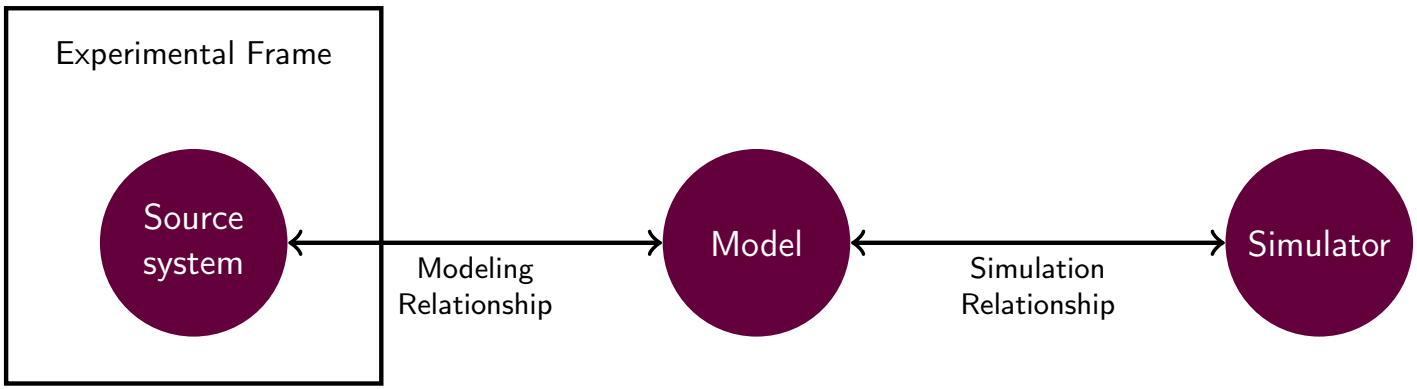
Modeling & Simulation (M&S) theory



1. What we want to model (source of observable data)
2. What we want to study (goal of the simulation + conditions)
3. Specifications to simulate the behaviour of the system
4. Computing system that can run the model
5. Validity of the model *w.r.t* physical system

Methodology

Modeling & Simulation (M&S) theory



1. What we want to model (source of observable data)
2. What we want to study (goal of the simulation + conditions)
3. Specifications to simulate the behaviour of the system
4. Computing system that can run the model
5. Validity of the model *w.r.t* physical system
6. Correct implementation of the simulation *w.r.t* model

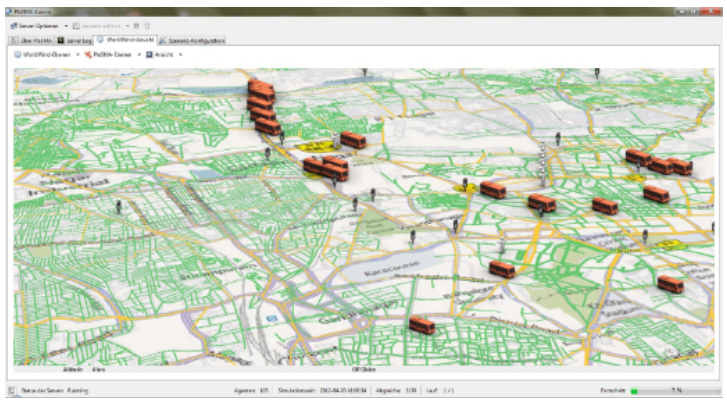
MultiAgent Based Simulation

A MultiAgent System to simulate a Dynamic System

- Agents $\leftrightarrow$ physical entities
- Actions and Interactions $\leftrightarrow$ behaviours

Examples

- Social simulation (*e.g.* opinion dynamics, road traffic, household)
- Physical phenomena (*e.g.* weather, geology, pandemy)
- Technical systems (*e.g.* electrical grid)





Why MABS?

Simulation of Complex Systems

When equations cannot capture everything

- ➜ Model (simpler) subsystems & combine them
- ➜ Study emerging phenomenons from individual choices
- ➜ Find parameters spaces in which stable situations appear

Explainable AI

- Analysing the microscopic level
- Modelling as close to reality as possible

Scalability

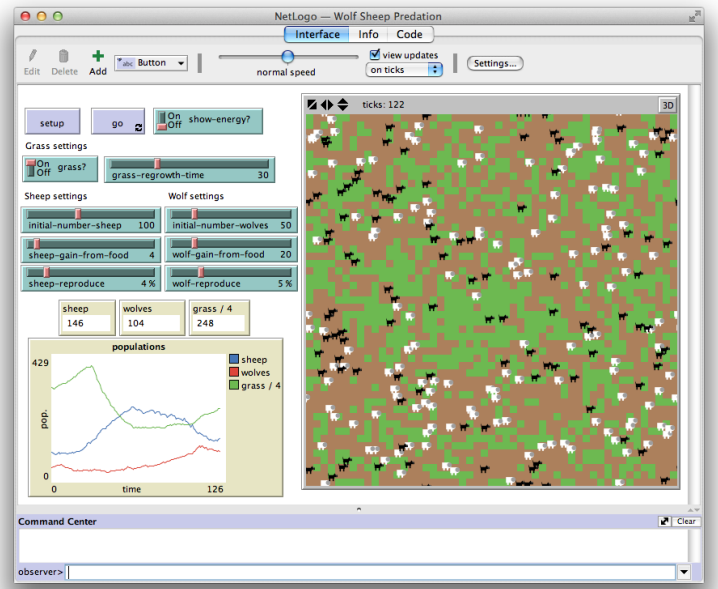
- Distributed computing & parallel simulation

Complements equation-based models!

The Prey-Predator example

Goal

Study how populations of preys & predators evolve in time



Voltera, 1926

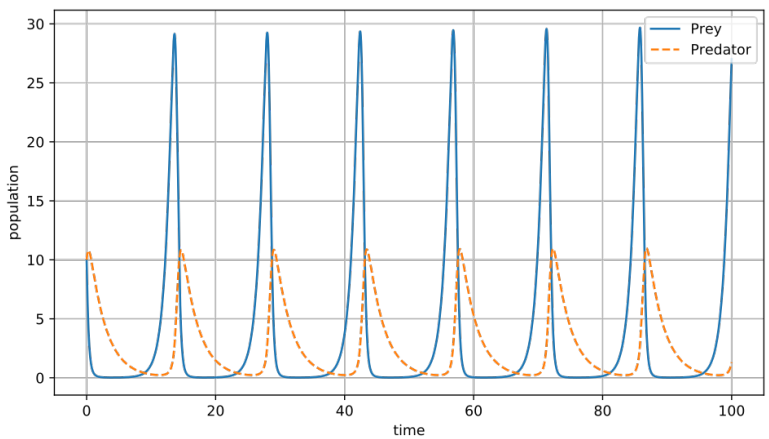
Equations Based Model (EBM)

$x =$ nb of preys, $y =$ nb of preds

$$\frac{dx(t)}{dt} = a.x(t) - b.x(t).y(t)$$

$$\frac{dy(t)}{dt} = c.x(t).y(t) - d.y(t)$$

with a, c reproduction rates
and b, d death rate



Remarks

Properties of EBM

Capture variability over time through interrelated equations

- ✓ Represents global variation of the populations (behaviours result)
- ✗ No explicit representation of individuals & interactions
- ✗ Large number of parameters sometimes hard to understand/manipulate

On the contrary, MABS

- ✓ Focus on entities and their interaction (study individual behaviours)
- ✗ Emergence of the global phenomenon not always easy to figure out
- ✗ Large number of parameters also...

Prey Predator in MABS

Agents

- **Prey**: move, reproduce, flee, die (opt. eat)
- **Predator**: move, reproduce, hunt (& eat), die

Environment

- Grid space
- (opt.) Grass growth

Prey Predator in MABS

Agents

- **Prey**: move, reproduce, flee, die (opt. eat)
- **Predator**: move, reproduce, hunt (& eat), die

Environment

- Grid space
- (opt.) Grass growth

Difficulty

Find the parameter values that reproduces the global phenomenon (a.k.a. stylised fact)

Design methodology

Use what we have seen for MAS + work incrementally!

Agents

- List the active entities in the system
- Identify their actions
- Modeling: choose the granularity level
- Define the interactions between these entities
- Possibly: define the organization

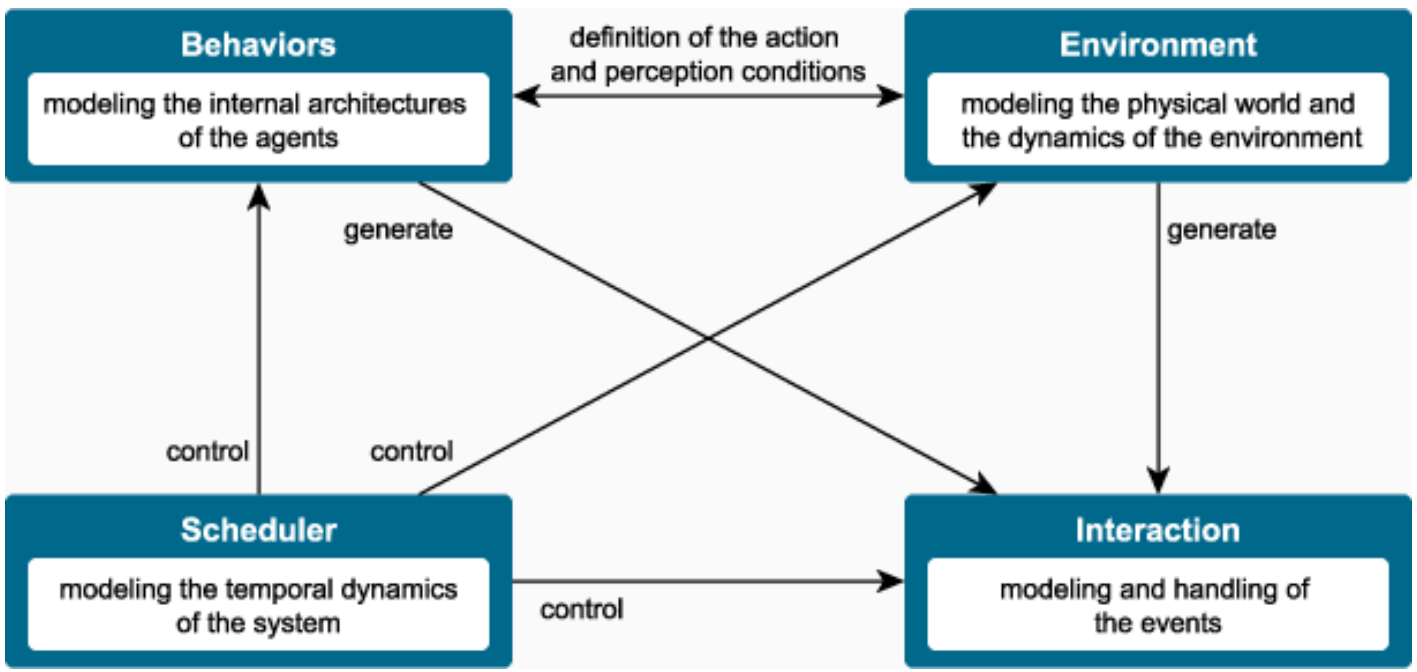
Environment

- List the shared resources (including spatialization)
- Modeling questions: discrete?, dynamic?, open?
- Technical questions: synchronicity?

Design methodology: summary

The four aspects of MABS models

Behavior: modelling of agents' deliberation processes
Environment: physical objects and dynamics of the environment
Scheduling: how time evolves and agents' runtimes interleave
Interaction: the result of actions



W. Ouerdane

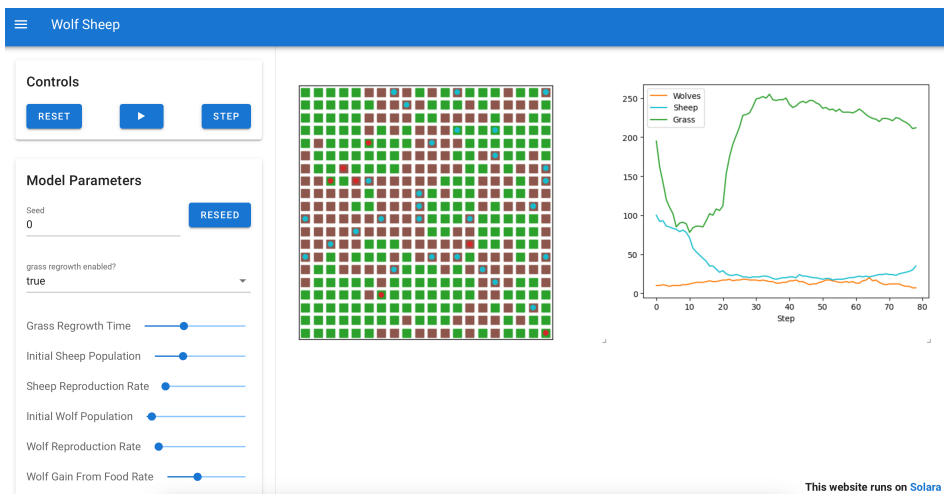
MESA

Mesa: Agent-Based Modeling in Python

https://mesa.readthedocs.io/

General principle

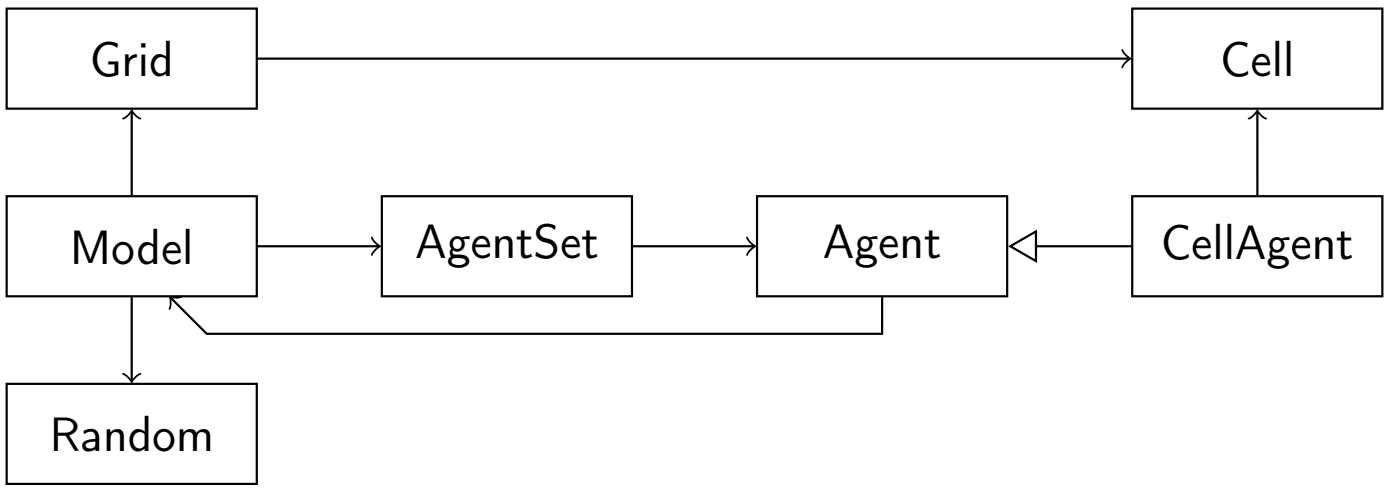
- Simulation-oriented MAS platform
- Built-in components : grid, agent scheduler, seed management
- Python API → access to python data analysis tools
- Netlogo-like web visualization interface
- Very active development



Mesa: main components

Classical MAS components

- Agents → class `mesa.Agent`
- Environment → class `mesa.Model`
- Runtime → class `mesa.AgentSet` + Model
- Spatial situation → classes in `mesa.discretespace`



Mesa: basics

The environment

- Subclass of mesa.Model

```
class MyModel(mesa.Model):
```

- Initialize with random seed

```
    def __init__(self,...):
        super().__init__(seed=None)
```

- Create agents

```
        SomeAgentClass.create_agents(model=self,n=...)
```

- Run the agents

```
    def step(self):
        self.agents.shuffle_do("procedural_loop")
```

- Run the model

```
model = MyModel(...)
model.run_for(number_of_turns)
```

Mesa: basics (cont.)

The agents

- Subclass of mesa.Agent

```
class MyAgent(mesa.Agent):
```

- Receives the model as parameter

```
    def __init__(self, model, ...):
        super().__init__(model)
```

- Define the procedural loop

```
    def procedural_loop(self):
       # generally named "step"
       # no other parameters!

       # code of the loop:
       ... # 1. perceive
       ... # 2. deliberate
       ... # 3. act
```

Minimal example

```python
import mesa

# Agent
class MyAgent(mesa.Agent):
    def __init__(self, model):
        super().__init__(model)

    def step(self):
        print("hello world")

# Environment + Runtime
class MyModel(mesa.Model):
    def __init__(self, n_agents):
        super().__init__()
        MyAgent.create_agents(self, n_agents) # AgentSet

        # runtime
    def step(self):
        self.agents.shuffle_do("step") # all agents -> 1 step

# Instanciate and run the model
model = MyModel(n_agents=5)
model.run_for(1)
```

Things to know about MESA

1. Encapsulation

Python has no OO access modifier

➜ Agents can access other agents' variables!

and this is **bad**!

Example:

```
class MyAgent(mesa.Agent):

    # init
    def __init__(self, model):
        super().__init__(model)
        self.my_secret_variable = self.random.randint(1,100)

    # step
    def step(self):
        # find any other agent...
        others = [a for a in self.model.agents if a is not self]
        if others:
            other = self.random.choice(cellmates)
            # ...and steal its secret variable!
            self.stolen_value = other.my_secret_variable
```

Things to know about MESA

1. Encapsulation

2. Procedural loop

Mesa does not force perception → deliberation → action

➜ You can write any code in the step method

and this is **very bad**!

Example:

```
def step(self):
    self.x = sum(a.y for a in self.model.schedule.agents)
    self.model.any_method(x)
    self.other_agent.any_method(x)
```

✗ Complexity n^2 with hidden perception

✗ 2 actions in one turn

Things to know about MESA

1. Encapsulation

2. Procedural loop

Mesa does not force perception → deliberation → action

➜ You can write any code in the step method

and this is **very bad too**!

✓ It would be better to write:

```
def step(self):
    self.perceive()   # update internal state
    a = self.decide() # select an action
    self.act(a)       # perform the action
```

... with ad-hoc methods

Things to know about MESA

1. Encapsulation

2. Procedural loop

3. Synchronous runtime

➜ Avoid long computation in the procedural loop

✗ All other agents are stopped during one's step

➜ If necessary, use external calls to some Thread

✓ Preferably in the environment

Things to know about MESA

1. Encapsulation
2. Procedural loop
3. Synchronous runtime
4. No direct communication
   ➜ All communications are done through variables in the environment (Model)

Direct communication

You can write your own message transport mechanism !

```
class Message:
    """Message structure"""
    REQUEST = 1
    REQUEST_DO = 2
    INFORM = 3
    def __init__(self, performatif: int, content: str, ...):
        self.performatif = performatif
        self.content = content
        ...

class MessageReceiver(mesa.discrete_space.CellAgent):
    """Agent that can receive messages"""
    def __init__(self, model):
        super().__init__(model)
        self.messages = []
    def notify(self, message: Message, sender):
        self.messages.append((message, sender))

class MyAgent(MessageReceiver):
    ...
    def perceive(self):
        return self.read_messages()
        # or any other message management
```

Practical work

Mesa Basics

Open the notebook

`session4_notebook.ipynb`

About this work

- Based on Mesa tutorials

  `https://mesa.readthedocs.io/latest/tutorials/0_first_model.html`

  `https://mesa.readthedocs.io/latest/tutorials/4_adding_space.html`

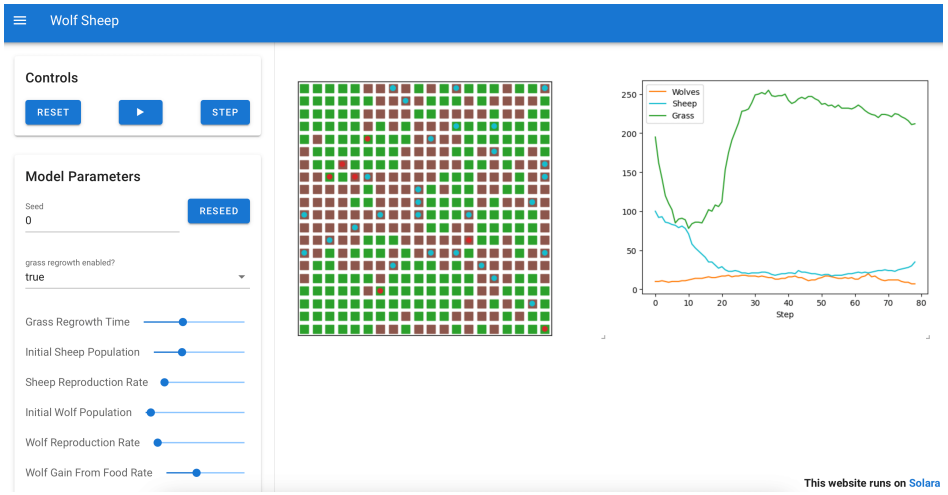
  `https://mesa.readthedocs.io/latest/tutorials/5_collecting_data.html`

- Can run in standalone or on Google colab
- Local package installation → must run in a virtualenv!

General idea

Simple simulation of inequality emergence in society (KISS model)

Mesa runtime visualization



General rules

1. Define agent portrayal
2. Define the model parameters = grid size + sliders
3. Instantiate the renderer

Does not work with Google Colab

Use local installation (jupyter or, better, standalone)

Mesa runtime visualization

Agent portrayal

A fonction that returns the visual representation of the agent

```
from mesa.visualization.components import AgentPortrayalStyle

def agent_portrayal(agent):
    # Default agents are small orange dots
    portrayal = AgentPortrayalStyle(size=50, color="tab:orange")
    # Wealthy agents are large blue dots
    if agent.wealth > 0:
        portrayal.update(("color", "tab:blue"), ("size", 100))
    return portrayal
```

Tips

- Use marker="^" to customize the agent's appearance
- Use alpha=0.8 for transparency and zorder=2 for superpositions (in multigrids)

Mesa runtime visualization

Grid renderer

The visual representation of the grid space

```
# instanciate the model
mm = MoneyModel(n=50, width=10, height=10)

# build the renderer
from mesa.visualization import SpaceRenderer

renderer = (
    SpaceRenderer(model=mm, backend="matplotlib")
    .setup_agents(agent_portrayal)
    .render()
)
```

Mesa runtime visualization

Model parameters

A dictionnary with values for all parameters: size of the grid, number of agents

```
model_params = {
    "n": {
        "type": "SliderInt",
        "value": 50,
        "label": "Number of agents:",
        "min": 10,
        "max": 100,
        "step": 1,
    },
    "width": 10,
    "height": 10,
}
```

Allows to change the model at runtime

Mesa runtime visualization

Method #1: solara server

➜ This is the default Mesa visualization method
https://mesa.readthedocs.io/latest/tutorials/6_visualization_basic.html

```
mm = MoneyModel(...)
def agent_portrayal(agent): ...
model_params = { ... }
rederer = ...

from mesa.visualization import SolaraViz

page = SolaraViz(
    mm,
    renderer,
    model_params=model_params,
    name="Boltzmann Wealth Model"
)

page
```

Command line `solara run this_file.py` to start the server

Mesa runtime visualization

Method #2: pygame library

➜ Home-made UPSaclay solution (Niccolo Angeli, 2026)
`https://github.com/Erocr/mesa_graphics`

Windows only (bug on pygame.font on Linux)

1. Download the library: `mesa_graphics` directory
2. Install pygame
3. Run as normal python file

```
mm = MoneyModel(n=50, width=10, height=10)
def agent_portrayal(agent): ...
model_params = { ... }

from mesa_graphics import MesaGraphics

page = MesaGraphics(
    mm,
    renderer,
    model_params=model_params,
    name="Boltzmann Wealth Model"
)
```

Mesa runtime visualization

Adding graphs

Dynamic representations based on data collections

- For Mesa Visualisation:

```
from mesa.visualization import make_plot_component
```

- For Mesa Graphics:

```
from mesa_graphics import make_plot_component
from mesa_graphics import FigureMatplotlib
from matplotlib.figure import Figure
```

Mesa runtime visualization

Adding graphs

Dynamic representations based on data collections

- For Mesa Visualisation:

```
from mesa.visualization import make_plot_component
```

- For Mesa Graphics:

```
from mesa_graphics import make_plot_component
from mesa_graphics import FigureMatplotlib
from matplotlib.figure import Figure
```

Simple example: draw Gini coefficient in time

```
GiniPlot = make_plot_component("Gini")

page = SolaraViz/MesaGraphics( mm, renderer,
    model_params=model_params,
    name="Boltzmann Wealth Model",
    components=[GiniPlot] # <- simply add the list of components
)
```

Mesa runtime visualization

Another example: histogram of agents' wealth

```
def Histogram(model):
    # Note: you must initialize a figure using this method instead of
    # plt.figure(), for thread safety purpose
    fig = Figure()
    ax = fig.subplots()
    wealth_vals = [agent.wealth for agent in model.agents]
    # Note: you have to use Matplotlib's OOP API instead of plt.hist
    # because plt.hist is not thread-safe.
    ax.hist(wealth_vals, bins=10)
    return FigureMatplotlib(fig)

page = MesaGraphics(
    mm,
    renderer,
    model_params=model_params,
    name="Boltzmann Wealth Model",
    components=[(Histogram, 2)] # <- draw the histogram on tab 2
)
```

Coming Next Practical Session MABS with Mesa or JADE